

“TUGAS PRATIKUM 6 SISTEM OPERASI”

**Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Operasi**

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:

NAMA : Reandra Khansa Faadihilah

NIM : 2410651060

KELAS : TI – C

Tanggal : 17-11-2025

UNIVERSITAS MUHAMMADIYAH JEMBER

2025-2026

DAFTAR ISI

DAFTAR ISI	2
Tugas 1 : Analisis Deadlock.....	3
Tugas 2 : Implementasi Pencegahan	4
Tugas 3 : Banker's Algorithm	7
Tugas 4 : Kasus Nyata (Real Case)	10

Tugas 1 : Analisis Deadlock

Pada tugas 1 kali ini kita belajar bagaimana cara memahami bagaimana deadlock bekerja dalam system operasi melalui simulasi sederhana menggunakan pthread mutex dengan menjalankan deadlock_simple.c

- Mengapa program mengalami deadlock?

Program mengalami deadlock karena kedua thread mengambil lock dalam urutan yang berbeda

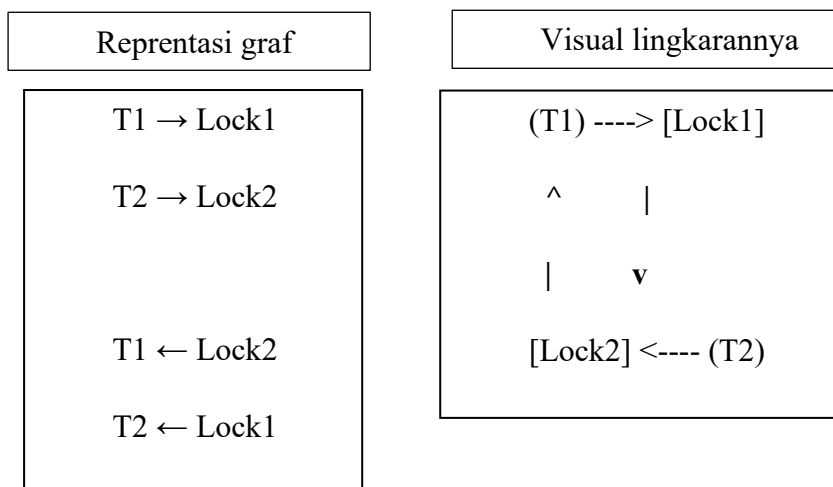
1. Thread 1: mengambil Lock 1 hingga menunggu lock 2
2. Thread 2 : mengambil Lock 2 hingga menunggu Lock 1

Keduanya saling menunggu sehingga tidak ada yang dapat melanjutkan. Karena kedua thread tidak pernah melepaskan lock yang sudah dimilikinya, program berhenti (hang)

- Kondisi deadlock apa saja yang terpenuhi?

1. Mutual Exclusion
Mutex hanya dapat dipegang satu thread pada satu waktu
2. Hold and Wait
Thread 1 memegang Lock1 sambil menunggu Lock2, dan sebaliknya
3. No. Preemption
Lock tidak dapat direbut paksa oleh thread lain
4. Circular Wait
T1 menunggu L2, T2 menunggu L1 sehingga membentuk lingkaran

- Gambar diagram resource allocation graph-nya!



Tugas 2 : Implementasi Pencegahan

Pada tugas ke-2 ini kita berfokus pada pencegahan terjadinya deadlock. Setelah memahami bagaimana deadlock terjadi pada tugas ke-1 kita memodifikasi program agar tidak lagi mengalami deadlock dengan dua teknik Lock ordering dan Try Lock dengan Retry

PRAKTIK

Modifikasi Lock Ordering

Memodifikasi file deadlock_simple.c

```
^ ~/SO6/Pratikum > gedit deadlock_simple.c c -gcc © 19:29

Open *deadlock_simple.c ~/SO6/Pratikum Save

1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
6 pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
7
8 // Selalu ambil lock dalam urutan yang sama: lock1 + lock2
9 void acquire_in_order() {
10     pthread_mutex_lock(&lock1);
11     printf("Mengambil Lock 1 ... \n");
12
13     pthread_mutex_lock(&lock2);
14     printf("Mengambil Lock 2 ... \n");
15 }
16
17 void release_in_order() {
18     pthread_mutex_unlock(&lock2);
19     pthread_mutex_unlock(&lock1);
20 }
21
22 void* thread1_func(void* arg) {
23     printf("Thread 1 mulai ... \n");
24     acquire_in_order();
25     printf("Thread 1 bekerja ... \n");
26     sleep(1);
27     release_in_order();
28     printf("Thread 1 selesai.\n");
29     return NULL;
30 }
31
32 void* thread2_func(void* arg) {
33     printf("Thread 2 mulai ... \n");
34     acquire_in_order();
35     printf("Thread 2 bekerja ... \n");
36     sleep(1);
37     release_in_order();
38     printf("Thread 2 selesai.\n");
39     return NULL;
40 }
41
42 int main() {
43     pthread_t t1, t2;
44
45     printf("== LOCK ORDERING: TANPA DEADLOCK ==\n");
46
47     pthread_create(&t1, NULL, thread1_func, NULL);
48     pthread_create(&t2, NULL, thread2_func, NULL);
49
50     pthread_join(t1, NULL);
51     pthread_join(t2, NULL);
52
53     printf("Program selesai.\n");
54     return 0;
55 }
```

COMPILE PROGRAM DAN OUTPUTNYA :

```
^ ~/S06/Pratikum > gcc deadlock_simple.c -o deadlock_lckordering -lpthread
^ ~/S06/Pratikum > ./deadlock_lckordering c -gcc © 19:39
=== LOCK ORDERING: TANPA DEADLOCK ===
Thread 1 mulai ...
Mengambil Lock 1 ...
Mengambil Lock 2 ...
Thread 1 bekerja ...
Thread 2 mulai ...
Thread 1 selesai.
Mengambil Lock 1 ...
Mengambil Lock 2 ...
Thread 2 bekerja ...
Thread 2 selesai.
Program selesai.
^ ~/S06/Pratikum > | c -gcc © 19:39
```

PENJELASAN

Program tersebut bekerja menghilangkan circular wait dengan memastikan semua thread mengambil locks dalam urutan yang sama sehingga deadlock tidak dapat terbentuk

Membuat dua helper `acquire_in_order()` dan `release_in_order()` dengan memastikan pengambilan lock 1 dan lock 2

Modifikasi Try-Lock dengan retry

Memodifikasi `deadlock_simple.c`

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread_func(void* arg) {
    int retry = 0;

    while (1) {
        printf("Thread 1 mencoba Lock 1 ... \n");
        pthread_mutex_lock(&lock1);

        printf("Thread 2 mencoba Lock 2 (trylock) ... \n");
        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1 berhasil mendapat semua lock \n");
            sleep(1);

            pthread_mutex_unlock(&lock2);
            pthread_mutex_unlock(&lock1);
            break;
        } else {
            printf("Thread 1 gagal Lock 2, melepas Lock 1 ... retry ... \n");
            pthread_mutex_unlock(&lock1);
            retry++;
            usleep(100000);
        }
    }

    printf("Thread 1 selesai (retry %d kali)\n", retry);
    return NULL;
}

void* thread2_func(void* arg) {
    int retry = 0;

    while (1) {
        printf("Thread 2 mencoba Lock 2 ... \n");
        pthread_mutex_lock(&lock2);

        printf("Thread 2 mencoba Lock 1 (trylock) ... \n");
        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2 berhasil mendapat semua lock \n");
            sleep(1);

            pthread_mutex_unlock(&lock1);
            pthread_mutex_unlock(&lock2);
            break;
        } else {
            printf("Thread 2 gagal Lock 1, melepas Lock 2 ... retry ... \n");
            pthread_mutex_unlock(&lock2);
            retry++;
            usleep(100000);
        }
    }

    printf("Thread 2 selesai (retry %d kali)\n", retry);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("== Try-Lock: Tanpa DEADLOCK ==\n");

    pthread_create(&t1, NULL, thread_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Program selesai \n");
    return 0;
}
```

COMPILE PROGRAM DAN OUTPUTNYA :

```
^ ~/S06/Pratikum > gcc deadlock_simple.c -o deadlock_trylock -lpthread
^ ~/S06/Pratikum > ./deadlock_trylock c -gcc © 19:40
== TRY-LOCK: TANPA DEADLOCK ==
Thread 1 mencoba Lock 1...
Thread 1 mencoba Lock 2 (trylock) ...
Thread 1 berhasil mendapat kedua lock!
Thread 2 mencoba Lock 2...
Thread 1 selesai (retry 0 kali)
Thread 2 mencoba Lock 1 (trylock) ...
Thread 2 berhasil mendapat kedua lock!
Thread 2 selesai (retry 0 kali)
Program selesai.
^ ~/S06/Pratikum > | c -gcc © 19:40
```

PENJELASAN

Thread melakukan `pthread_mutex_lock` pada lock pertama (blocking), lalu `pthread_mutex_trylock` pada lock kedua (non-blocking).

Jika trylock gagal, thread segera `pthread_mutex_unlock` lock pertama dan menunggu (`usleep`), lalu ulang loop.

Ketika trylock sukses, thread melakukan critical section, lalu melepaskan kedua lock dan keluar loop

Perbandingan Dari Ketiga Program

1. Program Deadlock Sederhana `deadlock_simple.c`

- Dua thread saling mengambil kunci (lock) dengan urutan berbeda.
- Akibatnya, mereka bisa saling menunggu dan program macet (deadlock).
- Tidak ada mekanisme pencegahan.

2. Program Lock Ordering `deadlock_prevention_ordering.c`

- Kedua thread mengambil kunci dengan urutan yang sama:
- Lock 1 dulu, baru Lock 2.
- Karena urutan sama, tidak ada yang saling menunggu.
- Program pasti aman dan tidak macet.

3. Program Try-Lock `deadlock_prevention_trylock.c`

- Thread mencoba mengambil lock kedua.
- Jika gagal, dia lepas lock pertama lalu coba lagi.
- Tidak akan deadlock, tetapi bisa perlu coba berkali-kali.

Tugas 3 : Banker's Algorithm

Pada tugas ke-3 kali ini kita membuat Banker Algorithm, Algoritma yang digunakan sistem operasi untuk menentukan apakah suatu permintaan resource aman atau tidak jika diberikan, dengan membuat program menghitung need, memeriksa safe state, dan menentukan apakah request dapat dikabulkan

PRAKTIK

Membuat file dengan nama banker_algorithm.c dengan text editor gedit

```
^ ~/S06/Tugas > gedit banker_algorithm.c  @ 19:48

*banker_algorithm.c
~/S06/Tugas

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define P 4
5 #define R 3
6
7 int available[R] = {5, 0, 3};
8
9 int maximum[P][R] = {
10     {7, 2, 3}, // P0
11     {1, 2, 2}, // P1
12     {9, 0, 2}, // P2
13     {2, 2, 2} // P3
14 };
15
16 int allocation[P][R] = {
17     {0, 1, 0}, // P0
18     {2, 0, 0}, // P1
19     {1, 0, 2}, // P2
20     {2, 1, 1} // P3
21 };
22
23 int need[P][R];
24
25 void calculate_need() {
26     for (int i = 0; i < P; i++)
27         for (int j = 0; j < R; j++)
28             need[i][j] = maximum[i][j] - allocation[i][j];
29 }
30
31 void print_status() {
32     printf("Current SYSTEM STATUS ==>\n");
33     printf("Available: ");
34     for (int j = 0; j < R; j++) printf("%d ", available[j]);
35     printf("\nAllocation:\n");
36     for (int i = 0; i < P; i++) {
37         printf("P%d: ", i);
38         for (int j = 0; j < R; j++) printf("%d ", allocation[i][j]);
39         printf("\n");
40     }
41     printf("\nNeed:\n");
42     for (int i = 0; i < P; i++) {
43         printf("P%d: ", i);
44         for (int j = 0; j < R; j++) printf("%d ", need[i][j]);
45         printf("\n");
46     }
47 }
48
49 bool is_safe() {
50     int work[R];
51     bool finish[P] = {false};
52     int safe_seq[P];
53     int count = 0;
54
55     for (int j = 0; j < R; j++) work[j] = available[j];
56     printf("Current safety seq:\n");
57     while (count < P) {
58         bool found = false;
59         for (int i = 0; i < P; i++) {
60             if (finish[i]) continue;
61             bool can = true;
62             for (int k = 0; k < R; k++) {
63                 if (need[i][k] > work[k]) { can = false; break; }
64             }
65             if (can) {
66                 printf("Can finish. Work: [");
67                 for (int j = 0; j < R; j++) printf("%d ", work[j]);
68                 printf("]\n");
69                 for (int j = 0; j < R; j++) work[j] += allocation[i][j];
70                 finish[i] = true;
71                 safe_seq[count] = i;
72                 count++;
73             }
74         }
75         if (count == P) {
76             printf("SUCCESS! A safe sequence found!\n");
77             return true;
78         }
79     }
80     printf("FAILURE! System is not in a safe state. Safe sequence: ");
81     for (int i = 0; i < P; i++) {
82         printf("%d ", safe_seq[i]);
83         if (i % 10 == 9) printf("\n");
84     }
85     printf("\n");
86     return false;
87 }
88
89 bool request_resource(int proc, int request[]) {
90     printf("Proc %d request: [", proc);
91     for (int j = 0; j < R; j++) printf("%d ", request[j]);
92     printf("]\n");
93
94     // Check request is valid
95     for (int j = 0; j < R; j++) {
96         if (request[j] > need[proc][j]) {
97             printf("Request exceeds maximum need!\n");
98             return false;
99         }
100     }
101
102     // Check request is available
103     for (int j = 0; j < R; j++) {
104         if (request[j] > available[j]) {
105             printf("FAILURE! Not enough resources available!\n");
106             return false;
107         }
108     }
109
110     // Portion allocation
111     for (int j = 0; j < R; j++) {
112         available[j] -= request[j];
113         allocation[proc][j] += request[j];
114         need[proc][j] -= request[j];
115     }
116
117     // Check safety
118     if (is_safe()) {
119         printf("SUCCESS! Request granted (system stays safe).\n");
120         return true;
121     } else {
122         printf("FAILURE! Request would lead to unsafe state. Rolling back.\n");
123         // Rollback
124         for (int j = 0; j < R; j++) {
125             available[j] += request[j];
126             allocation[proc][j] -= request[j];
127             need[proc][j] += request[j];
128         }
129     }
130 }
```

```

126 // Wellbark
127 printf("[DENY] Request would lead to unsafe state. Notting back.\n");
128 for (int j = 0; j < R; j++) {
129     available[j] += request[j];
130     allocation[proc][j] -= request[j];
131     need[proc][j] += request[j];
132 }
133 return false;
134 }
135
136 int main() {
137     printf("=== BANKER'S ALGORITHM - Tugas 3 ===\n");
138     calculate_need();
139     print_status();
140
141     // initial safety check
142     is_safe();
143
144     // Example request 1: safe request by P1
145     int r1[R] = {1, 0, 2};
146     request_resources(1, r1);
147     print_status();
148
149     // Example request 2: unsafe request by P2 (for demonstration)
150     int r2[R] = {3, 3, 0};
151     request_resources(2, r2);
152     print_status();
153
154     printf("\n=== PROGRAM FINISHED ===\n");
155     return 0;
156 }

```

COMPILE FILE DAN OUTPUTNYA

```

A ~/S06/Tugas > gcc banker_algorithm.c -o banker_algorithm c -gcc 8 10:53
A ~/S06/Tugas > ./banker_algorithm c -gcc 8 10:53
=== BANKER'S ALGORITHM - Tugas 3 ===

=== SYSTEM STATUS ===
Available: 5 4 3

Allocation:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 1 2 2
P2: 0 0 0
P3: 0 1 1

=== CHECKING SAFETY ===
P1 can finish. Mark: [5 4 3]
P2 can finish. Mark: [7 4 3]
P3 can finish. Mark: [10 4 5]
P0 can finish. Mark: [12 5 6]

[SUCCESS] System is in a SAFE state.
Safe sequence: P1 → P2 → P3 → P0

=== REQUEST from P1 ===
Request: 1 0 2

=== CHECKING SAFETY ===
P1 can finish. Mark: [4 4 1]
P2 can finish. Mark: [7 4 3]
P3 can finish. Mark: [10 4 5]
P0 can finish. Mark: [12 5 6]

[SUCCESS] System is in a SAFE state.
Safe sequence: P1 → P2 → P3 → P0
[GRANT] Request granted (system stays safe).

=== SYSTEM STATUS ===
Available: 4 4 1

Allocation:
P0: 0 1 0
P1: 3 0 2
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 0 2 0
P2: 0 0 0
P3: 0 1 1

=== REQUEST from P2 ===
Request: 3 3 0
[REJECT] Request exceeds maximum need!

=== SYSTEM STATUS ===
Available: 4 4 1

Allocation:
P0: 0 1 0
P1: 3 0 2
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 0 2 0
P2: 0 0 0
P3: 0 1 1

=== PROGRAM FINISHED ===
A ~/S06/Tugas >

```


PENJELASAN :

Program menghitung $\text{need} = \text{maximum} - \text{allocation}$.

Menampilkan status, memeriksa apakah sistem aman.

Melakukan 2 request contoh:

Request 1 dari P1: $\{1,0,2\} \rightarrow$ Jika aman, akan diberikan.

Request 2 dari P2: $\{3,3,0\} \rightarrow$ kemungkinan ditolak (Tidak aman) dan akan di-rollback.

Output akan menunjukkan safe sequence jika aman, atau pesan peringatan jika tidak

Tugas 4 : Kasus Nyata (Real Case)

Pada tugas ke-4 kali ini kita mensimulasikan proses dimana dua orang melakukan transfer uang antar rekening secara bersamaan, Jika dua proses mengambil kunci rekening secara berbeda urutan maka deadlock dapat terjadi, pada tugas kali ini kita menerapkan teknik pencegahan agar kasus tersebut dapat berjalan tanpa hang.

PRAKTIK

Membuat file **transfer_deadlock.c** dengan text editor gedit

Program sengaja dibuat deadlock untuk pengujian

```
^ ~/S06/Tugas > gedit transfer_deadlock.c
```

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <pthread.h>
4
5 typedef struct {
6     int id;
7     int balance;
8     pthread_mutex_t lock;
9 } Account;
10
11 Account acc1, acc2;
12
13 void transfer(Account *from, Account *to, int amount, const char *who) {
14     printf("As: trying to lock account %d (from)\n", who, from->id);
15     pthread_mutex_lock(&from->lock);
16     printf("As: locked account %d (from)\n", who, from->id);
17
18     usleep(100000); // give time for the other thread to lock the other account
19
20     printf("As: trying to lock account %d (to)\n", who, to->id);
21     pthread_mutex_lock(&to->lock);
22     printf("As: locked account %d (to)\n", who, to->id);
23
24     // perform transfer
25     if (from->balance > amount) {
26         *from->balance -= amount;
27         *to->balance += amount;
28         printf("As: transfer %d from %d to %d SUCCESS\n", who, amount, from->id, to->id);
29     } else {
30         printf("As: insufficient funds\n", who);
31     }
32
33     pthread_mutex_unlock(&to->lock);
34     pthread_mutex_unlock(&from->lock);
35 }
36
37 void personA(void* arg) {
38     transfer(&acc1, &acc2, 100, "PersonA");
39     return NULL;
40 }
41
42 void personB(void* arg) {
43     transfer(&acc2, &acc1, 200, "PersonB");
44     return NULL;
45 }
46
47 int main() {
48     acc1.id = 1; acc1.balance = 1000; pthread_mutex_init(&acc1.lock, NULL);
49     acc2.id = 2; acc2.balance = 1000; pthread_mutex_init(&acc2.lock, NULL);
50
51     pthread_t t1, t2;
52     printf("=== Transfer SIM (NEEDNOT BE DEADLOCK) ===\n");
53     pthread_create(&t1, NULL, personA, NULL);
54     pthread_create(&t2, NULL, personB, NULL);
55
56     pthread_join(t1, NULL);
57     pthread_join(t2, NULL);
58
59     printf("Final balances: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);
60     return 0;
61 }

```

COMPILE DAN OUTPUNYA :

```

A ~/S06/Tugas > gcc transfer_deadlock.c -o transfer_deadlock c -gcc @ 19:57
A ~/S06/Tugas > ./transfer_deadlock c -gcc @ 19:57
=== TRANSFER SIM (BERPOTENSI DEADLOCK) ===
PersonA: trying to lock account 1 (from)
PersonA: locked account 1 (from)
PersonB: trying to lock account 2 (from)
PersonB: locked account 2 (from)
PersonA: trying to lock account 2 (to)
PersonB: trying to lock account 1 (to)
^C
A ~/S06/Tugas > c -gcc @ 19:57

```

PENJELASAN :

Program ini sengaja dibuat untuk mengalami deadlock dan akan macet ketika dijalankan bersamaan oleh kedua thread.

Thread PersonA mengunci acc1 → sleep → coba kunci acc2

Thread PersonB mengunci acc2 → sleep → coba kunci acc1

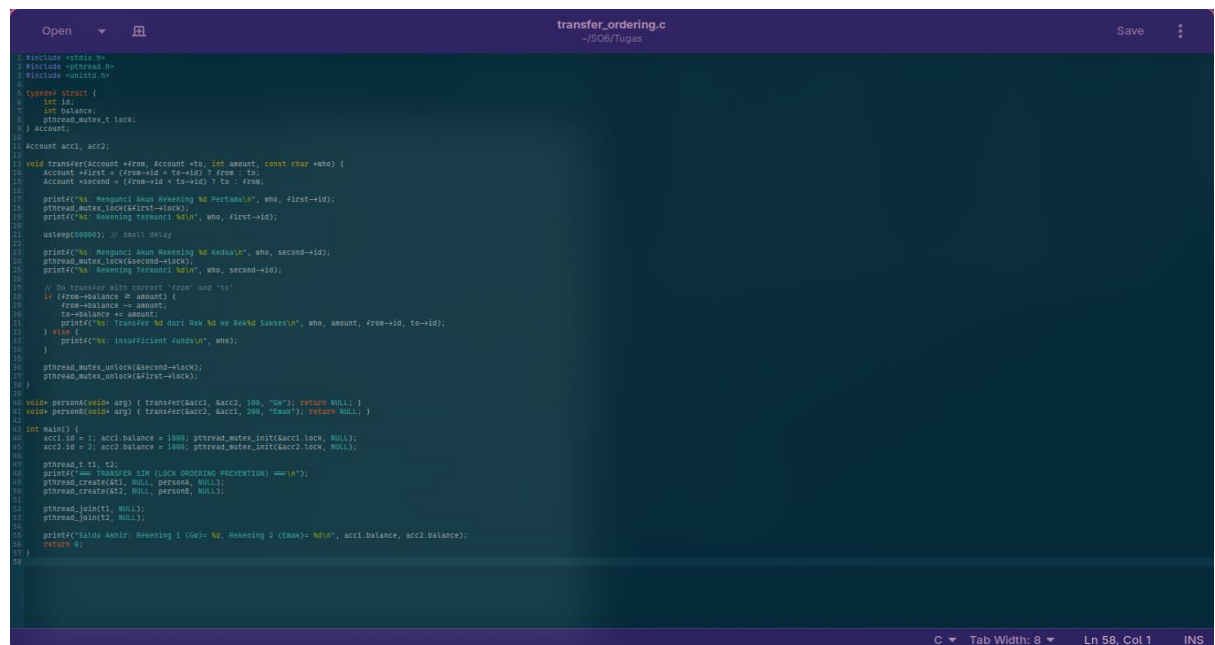
→ Terjadi circular wait → kedua thread saling menunggu selamanya.

Memodifikasi Program dengan mencegah deadlock sepenuhnya dengan lock ordering

PRAKTIK

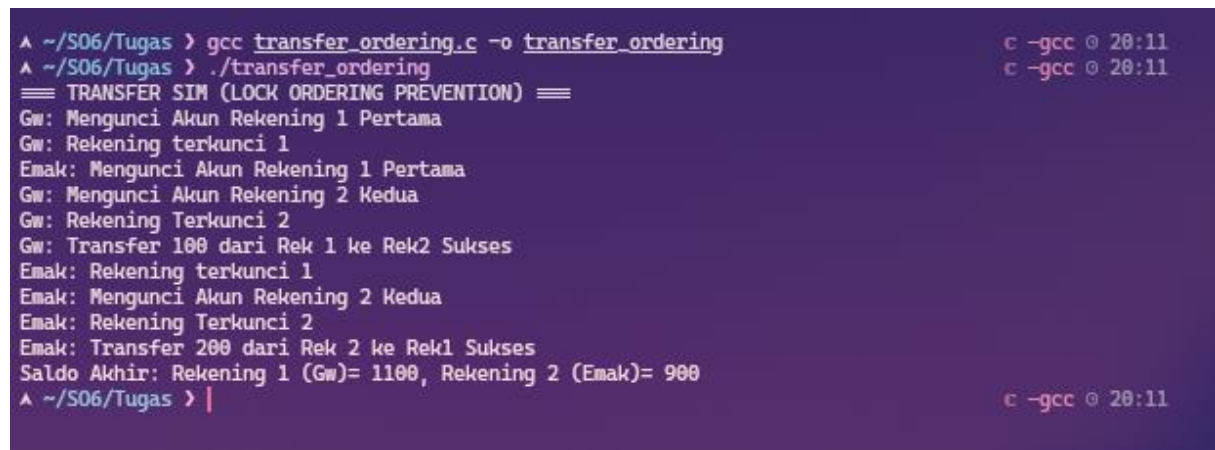
Membuat program dengan nama **transfer_ordering.c**

```
^ ~/S06/Tugas > gedit transfer_ordering.c
```



```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 typedef struct {
6     int id;
7     int balance;
8     pthread_mutex_t lock;
9 } Account;
10
11 Account acc1, acc2;
12
13 void transfer(Account *from, Account *to, int amount, const char *who) {
14     Account *first = (from->id < to->id) ? from : to;
15     Account *second = (from->id < to->id) ? to : from;
16
17     printf("%s: Mengunci Akun Rekening 1 Pertama\n", who, first->id);
18     pthread_mutex_lock(&first->lock);
19     printf("%s: Rekening terkunci %d\n", who, first->id);
20
21     usleep(50000); // small delay
22
23     printf("%s: Mengunci Akun Rekening 2 Kedua\n", who, second->id);
24     pthread_mutex_lock(&second->lock);
25     printf("%s: Rekening terkunci %d\n", who, second->id);
26
27     // Do transfer with correct 'from' and 'to'
28     if (from->balance >= amount) {
29         from->balance -= amount;
30         to->balance += amount;
31         printf("%s: Transfer %d dari Rek %d ke Rek %d Sukses\n", who, amount, from->id, to->id);
32     } else {
33         printf("%s: Insufficient funds\n", who);
34     }
35
36     pthread_mutex_unlock(&second->lock);
37     pthread_mutex_unlock(&first->lock);
38 }
39
40 void *personA(void *arg) { transfer(&acc1, &acc2, 100, "Gw"); return NULL; }
41 void *personB(void *arg) { transfer(&acc2, &acc1, 200, "Emak"); return NULL; }
42
43 int main() {
44     acc1.id = 1; acc1.balance = 1000; pthread_mutex_init(&acc1.lock, NULL);
45     acc2.id = 2; acc2.balance = 1000; pthread_mutex_init(&acc2.lock, NULL);
46
47     pthread_t t1, t2;
48     printf("=== TRANSFER SIM (LOCK ORDERING PREVENTION) ===\n");
49     pthread_create(&t1, NULL, personA, NULL);
50     pthread_create(&t2, NULL, personB, NULL);
51
52     pthread_join(t1, NULL);
53     pthread_join(t2, NULL);
54
55     printf("Saldo Akhir: Rekening 1 (Gw)= %d, Rekening 2 (Emak)= %d\n", acc1.balance, acc2.balance);
56     return 0;
57 }
```

COMPILE DAN HASILNYA :



```
^ ~/S06/Tugas > gcc transfer_ordering.c -o transfer_ordering
^ ~/S06/Tugas > ./transfer_ordering
=== TRANSFER SIM (LOCK ORDERING PREVENTION) ===
Gw: Mengunci Akun Rekening 1 Pertama
Gw: Rekening terkunci 1
Emak: Mengunci Akun Rekening 1 Pertama
Gw: Mengunci Akun Rekening 2 Kedua
Gw: Rekening Terkunci 2
Gw: Transfer 100 dari Rek 1 ke Rek2 Sukses
Emak: Rekening terkunci 1
Emak: Mengunci Akun Rekening 2 Kedua
Emak: Rekening Terkunci 2
Emak: Transfer 200 dari Rek 2 ke Rek1 Sukses
Saldo Akhir: Rekening 1 (Gw)= 1100, Rekening 2 (Emak)= 900
^ ~/S06/Tugas > |
```

PENJELASAN

Program ini mencegah deadlock dengan teknik lock ordering (mengunci akun dengan id yang lebih dahulu)

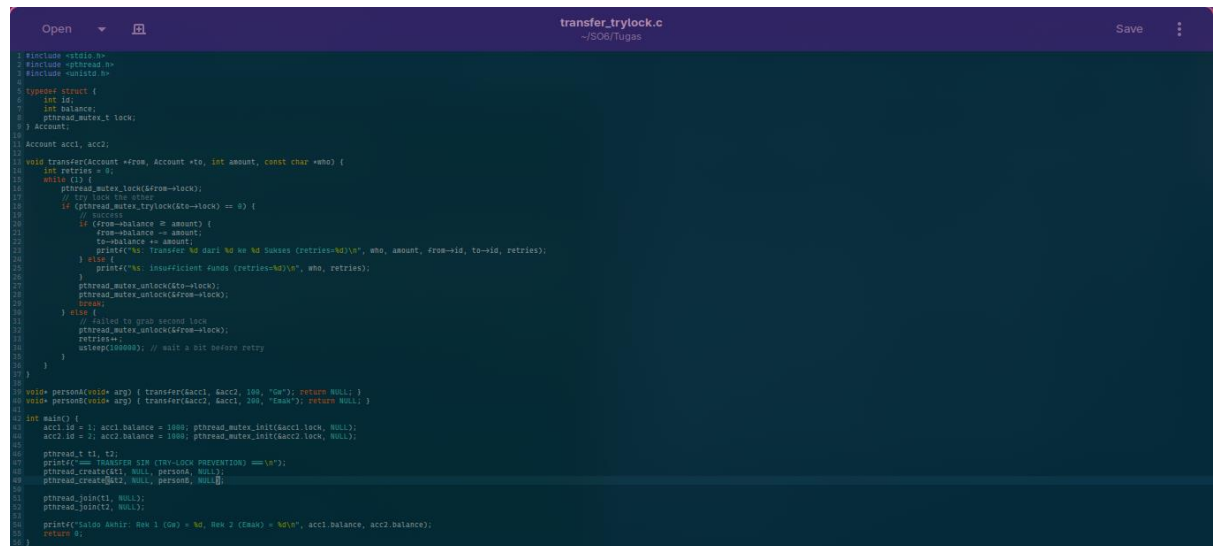
Kedua thread melakukan transfer silang acc1 ke acc2 dan acc2 ke acc1 tanpa adanya circular wait dengan mengurutkan penguncian selalu sama

Membuat program dengan mencegah deadlock dengan teknik trylock

PRAKTIK

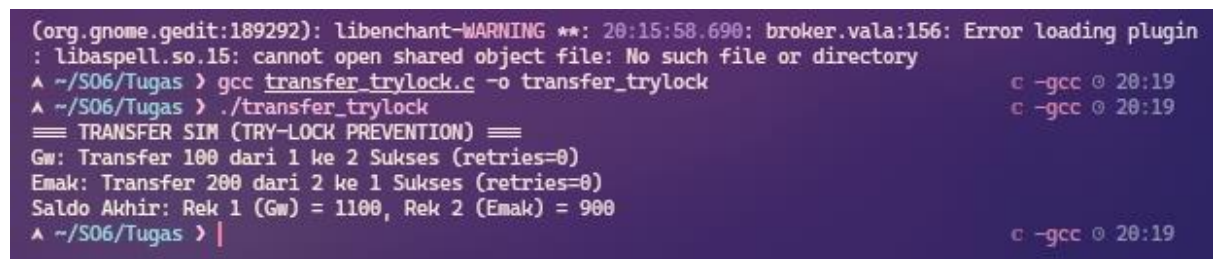
Membuat program dengan nama transfer_trylock.c

```
^ ~/S06/Tugas > gedit transfer_trylock.c
```



```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 typedef struct {
6     int id;
7     int balance;
8     pthread_mutex_t lock;
9 } Account;
10
11 Account acc1, acc2;
12
13 void transfer(Account *from, Account *to, int amount, const char *who) {
14     int retries = 0;
15     while (1) {
16         pthread_mutex_lock(&from->lock);
17         if (pthread_mutex_trylock(&to->lock) == 0) {
18             // Success
19             if (from->balance < amount) {
20                 from->balance -= amount;
21                 to->balance += amount;
22                 printf("%s: Transfer %d dari %d ke %d Sukses (retries=%d)\n", who, amount, from->id, to->id, retries);
23             } else {
24                 printf("%s: Insufficient funds (retries=%d)\n", who, retries);
25             }
26             pthread_mutex_unlock(&to->lock);
27             pthread_mutex_unlock(&from->lock);
28             if (retries < 10) {
29                 // Failed to grab second lock
30                 pthread_mutex_unlock(&from->lock);
31                 usleep(100000); // wait a bit before retry
32             }
33         }
34     }
35 }
36
37 void personA(void *arg) { transfer(&acc1, &acc2, 100, "Gw"); return NULL; }
38 void personB(void *arg) { transfer(&acc2, &acc1, 200, "Emak"); return NULL; }
39
40 int main() {
41     acc1.id = 1; acc1.balance = 1000; pthread_mutex_init(&acc1.lock, NULL);
42     acc2.id = 2; acc2.balance = 1000; pthread_mutex_init(&acc2.lock, NULL);
43
44     pthread_t t1, t2;
45     printf("Transfer SIM (TRY-LOCK PREVENTION) ==>\n");
46     pthread_create(&t1, NULL, personA, NULL);
47     pthread_create(&t2, NULL, personB, NULL);
48     pthread_join(t1, NULL);
49     pthread_join(t2, NULL);
50
51     printf("Saldo Akhir: Rek 1 (Gw) = %d, Rek 2 (Emak) = %d\n", acc1.balance, acc2.balance);
52     return 0;
53 }
```

HASIL & COMPILENYA :



```
(org.gnome.gedit:189292): libenchant-WARNING **: 20:15:58.690: broker.vala:156: Error loading plugin
: libaspell.so.15: cannot open shared object file: No such file or directory
^ ~/S06/Tugas > gcc transfer_trylock.c -o transfer_trylock c -gcc @ 20:19
^ ~/S06/Tugas > ./transfer_trylock c -gcc @ 20:19
== TRANSFER SIM (TRY-LOCK PREVENTION) ==
Gw: Transfer 100 dari 1 ke 2 Sukses (retries=0)
Emak: Transfer 200 dari 2 ke 1 Sukses (retries=0)
Saldo Akhir: Rek 1 (Gw) = 1100, Rek 2 (Emak) = 900
^ ~/S06/Tugas > | c -gcc @ 20:19
```

PENJELASAN :

Pada program ke-3 ini mencegah adanya deadlock dengan teknik trylock + rollback + retry

Thread mengunci akun pertama dengan `pthread_mutex_lock`

Mencoba mengunci akun kedua dengan `pthread_mutex_trylock`

Jika gagal → lepas lock pertama → tunggu sebentar → coba lagi dari awal

Dengan adanya program `transfer_trylock.c` tidak pernah memegang satu lock sambil menunggu lock lain secara permanen